ARTICLE

Design of a high performance FFT processor based on FPGA

CHU CHAO, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

ZHANG QIN, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

XIE YINGKE, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

HAN CHENGDE, Institute of Computing Technology Chinese Academy of Sciences, Beijing, Beijing, China

Published: 18 January 2005

ASPDAC05: Asia and South Pacific Design Automation Conference
January 18 - 21, 2005
Shanghai, China

Conference Sponsors:
SIGDA

Design of a High Performance FFT Processor Based on FPGA

Chu Chao[1, 2] Zhang Qin[1, 2] Xie Yingke[1] Han Chengde[1]

1. Institute of Computing Technology, CAS, Beijing 100080
2. Graduate School of the Chinese Academy of Sciences, Beijing, 100039
{chuchao, zhangqin, ykxie, han}@ict.ac.cn

***Abstract*—The design method of a real-time FFT processor is presented. By optimizing algorithm of memory mapping and generation of twiddle factors, a radix-4 butterfly can be calculated in one clock cycle. An approach to adaptive overflow control is also introduced to avoid overflow without interrupting the computing pipeline. The design is implemented on a FPGA chip and achieves the operating frequency at 127 MHz. It can complete a complex 1024-point FFT within 10.1 μs.**

***Index Terms*—FFT processor, FPGA, address generation, overflow control**

I. INTRODUCTION

The Discrete Fourier Transform (DFT), which facilitates the efficient transformation between the time domain and the frequency domain for a sampled signal, is one of the most used algorithms in digital signal processing. Among these applications the long-length DFT computation is rather time-consuming, hence high performance Fast Fourier Transform (FFT) processor is necessary to meet many real-time requirements, e.g., DVB-T and DAB. But at present most commercial and academic FFT processors take dozens of μs to compute a 1024-point complex transform [1], hence and therefore design of a high performance FFT processor become a key issue of real-time system. At present the approaches to improve the performance of FFT processor: to raise work frequency and to enhance functional parallel [2][3]. With highly parallel computing units, the speed of data access becomes the bottleneck of system [4].

In the past twenty years, Field Programmable Gate Array (FPGA) and Programmable Logic Device (PLD) have developed rapidly and at current stage digital signal processors based on FPGA are applied in most areas of signal processing.. Compared with traditional ASIC design flow, design based on FPGA has the advantages of flexibility and time to market objective.

In this paper, we develop a soft-core design to implement a FFT processor based on FPGA. The next section reviews necessary backgrounds used in the development of the paper including FFT algorithm and lifting scheme. Section III presents the architecture of the fix-point FFT processor with overflow control. By the methods of optimizing algorithm of memory mapping and generation of twiddle factors, this FFT processor with a better data throughput can calculate a radix-4 butterfly in one clock cycle. Furthermore, we propose an overflow control approach according to the result of butterfly calculation without interrupting the pipeline of calculation unit. Section IV discusses simulation result of FFT processor on a FPGA chip and comparison with other designs. Finally, Section V concludes the whole paper.

II. ALGORITHM

A. FFT Algorithm

The DFT for a N-point data sequence of input signal, $\{x(n)\}, x = 0,1,...,N-1$, is defined as

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk}, \ for\ k = 0,1,\cdots,N-1 \qquad (1)$$

where

$$W_N = e^{-j\cdot\frac{2\pi}{N}}, \ for\ N = 2^n.$$

Pursuant to (1), direct computation of a N-point DFT requires $O(N^2)$ complex number multiplications and complex number additions, which are a large-scale computation. In 1965, J. W. Cooley and J. W. Tukey proposed a fast algorithm of DFT with the property of in-place memory addressing strategy, which is convenient to be implemented in hardware; therefore, most of current DFT processors are based on this algorithm [5].

There are various algorithms to implement FFT, such as radix-2, radix-4 and split-radix with arbitrary sizes. Radix-2 algorithm is the simplest one, but its calculation of addition and multiplication is more than radix-4's. Through being more efficient than radix-2, radix-4 only can process 4^n-point FFT. Split-radix has the advantage of less calculation than radix-2 and radix-4, but it is difficult to implement L-shaped butterfly function unit in split-radix. Taking account of algorithm complexity and hardware cost, in this paper, we implement the radix-4 algorithm in DIF (Decimation in Frequency). Sampled at $N = 4^r$ points, (1) is decimated in frequency accordance with this mode:

$$X(k) = \sum_{n=0}^{N/4-1} x(n) W_N^{nk} + \sum_{n=N/4}^{N/2-1} x(n) W_N^{nk} + \sum_{n=N/2}^{3N/4-1} x(n) W_N^{nk} + \sum_{n=3N/4}^{N-1} x(n) W_N^{nk} \qquad (2)$$

Let us assume that $k = 4m$, $k = 4m+2$, $k = 4m+1$ and $k = 4m+3$, where $m = 0,1,...,N/4-1$, we get

$$X(4m) = \sum_{n=0}^{N/4-1} \left[(x(n) + x(n+N/2)) + (x(n+N/4) + x(n+3N/4)) \right] \cdot W_{N/4}^{nm} \qquad (3a)$$

$$X(4m+2) = \sum_{n=0}^{N/4-1} \left[(x(n) + x(n+N/2)) - (x(n+N/4) + x(n+3N/4)) \right] \cdot W_N^{2n} \cdot W_{N/4}^{nm} \qquad (3b)$$

$$X(4m+1) = \sum_{n=0}^{N/4-1} \left[(x(n) - x(n+N/2)) - j(x(n+N/4) - x(n+3N/4)) \right] \cdot W_N^{n} \cdot W_{N/4}^{nm} \qquad (3c)$$

$$X(4m+3) = \sum_{n=0}^{N/4-1} \left[(x(n) - x(n+N/2)) + j(x(n+N/4) - x(n+3N/4)) \right] \cdot W_N^{3n} \cdot W_{N/4}^{nm} \qquad (3d)$$

The above equation denotes the basic DIF operation of radix-4 FFT, which is generally called butterfly calculation. Hence, the whole calculation complexity of this algorithm is reduced to $O(N \log_4 N)$.

B. Lifting Scheme

Initially, the lifting scheme has been proposed to be a tool in constructing wavelets and Perfect Reconstruction (PR) filter banks [6]. The resulting transforms are shown to be simple and invertible, even though the lifting coefficients are quantized and power-adaptable. Since all the twiddle factors appearing in the FFT structure are complex numbers with magnitude one, that is, every factors can be represented as $e^{j\theta}$, where θ is some real number. Then, a twiddle factor W in complex form can be expressed as

$$W = \cos\theta + j\sin\theta = c + js \tag{4}$$

In general, a complex multiplication is decomposed into four real multiplications. Here, let another complex number be $X = x + jy$, and then the product of these two complex numbers is

$$P = W \cdot X = (\mathrm{c} + j\,\mathrm{s})(x + jy) \tag{5}$$

In the vector-matrix form for convenience

$$P = \begin{bmatrix} 1 & j \end{bmatrix} \begin{bmatrix} \mathrm{c} & -\mathrm{s} \\ \mathrm{s} & \mathrm{c} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \tag{6}$$

Assuming that $c^2 + s^2 = 1$ with $s \neq 0$ (if $s = 0$, then $W = 1$ *or* -1, which needs not to be quantized), by decomposing into three lifting steps we can get

$$\begin{bmatrix} \mathrm{c} & -\mathrm{s} \\ \mathrm{s} & \mathrm{c} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ s/c & 1 \end{bmatrix} \begin{bmatrix} 1 & -\mathrm{s}\,c \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \mathrm{c} & 0 \\ 0 & 1/c \end{bmatrix} \tag{7}$$

We can also do it without scaling with three lifting steps as (here assuming $s \neq 0$ too)

$$\begin{bmatrix} \mathrm{c} & -\mathrm{s} \\ \mathrm{s} & \mathrm{c} \end{bmatrix} = \begin{bmatrix} 1 & (c-1)/s \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ s & 1 \end{bmatrix} \begin{bmatrix} 1 & (c-1)/s \\ 0 & 1 \end{bmatrix} \tag{8}$$

This corresponds to the well-known fact in geometry that a rotation can always be written as three shears. Here are two advantages of this factorization scheme [7]. Firstly the number of real multiplications is reduced from four to three, although, the number of real additions is increased from two to three. Secondly, it allows for quantization of lifting coefficients and result of each multiplication without destroying the PR property.

III. Design of FFT Processor

A. Architecture

The research objective of this paper is to enhance the performance of FFT processor by optimizing structure implementation and simplifying hardware design. Here, we improve the throughput of datapath by optimizing the design of memory mapping and the generation of twiddle factor. In addition to lifting scheme implemented in this processor, we propose an adaptive overflow control approach with less hardware cost and higher computing precision.

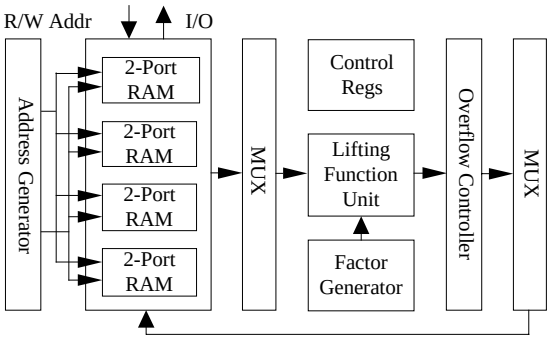

Fig. 1. Hierarchical block diagram of FFT processor

Fig. 1 shows the hierarchical block diagram of this FFT processor, which consists of several parts: four 2-port BlockRAM to store data; address generator which outputs four read addresses and write addresses for I/O ports per cycle; butterfly calculation unit which comprises lifting function unit and generator of twiddle factors, can calculate a radix-4 butterfly per cycle; units of data interchange which rearrange data in right sequence; overflow controller which controls dynamically shift of fix-point operand according to result of butterfly calculation. The detailed implementations of function units will be discussed in the next subsections.

B. Method of Address Mapping

Each butterfly unit of radix-4 FFT needs four operands per cycle, and then produces four results, which proves that parallel access to data is crucial issue for system efficiency. D. Cohen [8] and Y. T. Ma [9], etc., proposed several approaches of address mapping, which are not appropriate to the structure of single butterfly unit. In this paper, we illustrate a method of address mapping and generation with in-place memory addressing strategy on the basis of [2] and implement this method in DIF of radix-4 FFT. Now let us discuss the method of address mapping.

The address of every operand can be expressed as

$$A = \sum_{i=0}^{r-1} 4^i \cdot A(i),\ A(i) = 0,1,2,3;\ r = \log_4 N \tag{9}$$

In another form

$$A = 4 \cdot \sum_{i=1}^{r-1} 4^{i-1} \cdot A(i) + A(0) \tag{10}$$

Assuming that

$$B = 4 \cdot \sum_{i=1}^{r-1} 4^{i-1} \cdot A(i) + \left(\sum_{i=0}^{r-1} A(i) \right) \bmod 4 \tag{11}$$

It's obvious that address B is corresponding to address A one by one in this area: $[0, N)$.

Let $m = \sum_{i=1}^{r-1} 4^{i-1} \cdot A(i)$, $b = \left(\sum_{i=0}^{r-1} A(i) \right) \bmod 4$.

Thus B can be replaced by m and b as

$$B = 4m + b \tag{12}$$

It can be found that address A is mapping into 4 memory banks, where b is the number of bank and m is the address in bank; that is to say, the four operands of radix-4 calculation are mapped into different memory banks to avoid conflict.

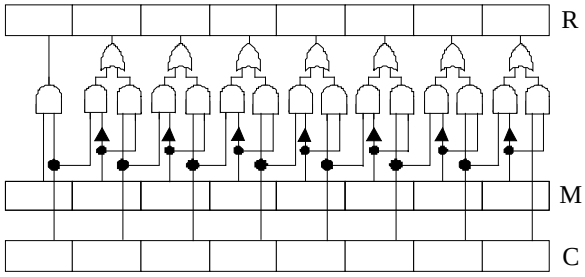

Fig. 2. Implementation of address generator

C. Address Generator

Considering the features of address mapping method and radix-4 DIF algorithm, we propose an approach of address generation, which is easy to be implemented and extended in hardware. We briefly describe the detail of implementation now. Corresponding to the butterfly calculation of the p $(p = 0, 1, \cdots, r-1)$ level, here only the $r-p-1$ bits of four operands are different: 0, 1, 2 and 3. These addresses in banks of these operands are described as:

$$m = \sum_{i=r-p}^{r-1} 4^{i-1} \cdot A(i) + 4^{r-p-2} \cdot A(r-p-1) + \sum_{i=1}^{r-p-2} 4^{i-1} \cdot A(i) \qquad (13)$$

Let $A(r-p-1) = 0, 1, 2, 3$ respectively, then

$$m_0 = \sum_{i=r-p}^{r-1} 4^{i-1} \cdot A(i) + \sum_{i=1}^{r-p-2} 4^{i-1} \cdot A(i) \qquad (14a)$$

$$m_1 = m_0 + 1 \times 4^{r-p-2} \qquad (14b)$$

$$m_2 = m_0 + 2 \times 4^{r-p-2} \qquad (14c)$$

$$m_3 = m_0 + 3 \times 4^{r-p-2} \qquad (14d)$$

In order to compute address m of the above equations quickly, here a counter of 16-bit width is implemented in hardware to denote $\sum_{i=r-p-1}^{r-2} 4^{i-1} \cdot A(i+1) + \sum_{i=1}^{r-p-2} 4^{i-1} \cdot A(i)$, then we insert $A(r-p-1)$ into the $(r-p-1)$ bit of counter to get m_i, which needs three 16-bit width registers: counter C, shift control register M and result register R. As shown in Fig. 2, C is set to zero initially and added by one after every butterfly calculation; M is set to zero initially too and right-shifted two bits after every level operation, also the high two bits are filled with 11; R has initial value m_0. Pursuant to (14), we can get another three addresses by setting some value to the $(r-p-1)$ bit.

Here, modulus operation of bank number b can be implemented by using half-adder; since $A(i)$ is 2-bit width for radix-4 FFT, we can compute b with two half-adders.

D. Hardware Implementation of Lifting Structure

As mentioned in the previous subsection, a complex multiplication can be decomposed into 4 real multiplications. Generally speaking, it needs more hardware resource, such as multipliers and registers to execute complex multiplications in parallel mode; On the other hand, execution in sequence mode wastes more cycles and save some hardware costs.

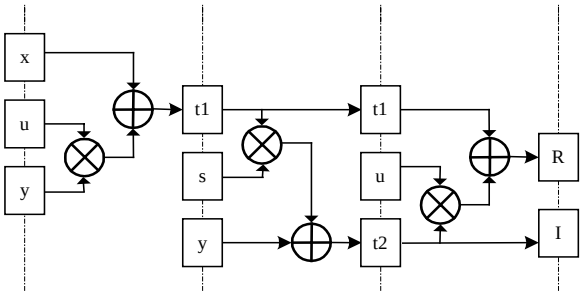

Fig. 3. Pipeline structure of lifting scheme

Considering the fact of twiddle factors with magnitude one in FFT, we implement complex multiplication in terms of lifting scheme instead of conventional method [7]. Notice that the three multiplications must be executed in sequence via the lifting scheme as described in (8).

Here, we introduce a pipeline structure: it follows from (8) that implements a lifting scheme structure in sequence, which is designed in pipeline mode to avoid the overhead of clock frequency. Fig. 3 shows the pipeline structure of this lifting scheme (here, $u = (c-1)/s$).

The above design based on lifting scheme needs three multipliers and three adders instead of four multipliers and two adders in direct calculation method. It should also be noted that here it needs to store $(c-1)/s$ instead of c in ROM in advance, which has no additional timing costs since both $(c-1)/s$ and c can be pre-computed to store in ROM.

E. Adaptive Overflow Control

In general, FFT processors deal with overflow problem by virtue of limiting datapath width, which directly truncates overflowing part of data; hence this method is not appropriate to most of applications because of bad precision. A dynamic approach of overflow control can be reached by selecting the input mode of the $i+1$ level according to calculation result of the i level. This approach ensures that no overflow takes place in the whole calculation period, however, it needs to interrupt the pipeline of computing with a low efficiency. In this paper, we propose a novel approach without interrupting pipeline: to select the writeback mode of the $i+1$ level according to the writeback result of the i level. Therefore, this overflow control method keeps pipeline efficiency close to 100% in the whole calculation period and has less right-shift operations.

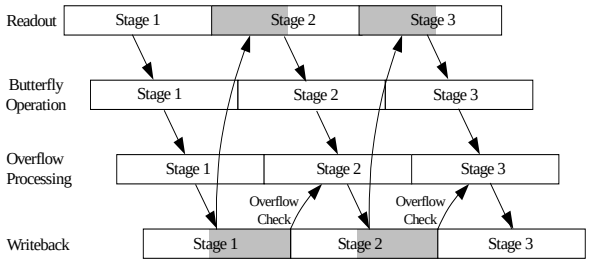

Fig. 4. Diagram of FFT processor's pipeline

TABLE I
PERFORMANCE COMPARISON WITH OTHER FIX-POINT FFT PROCESSORS [1] [10] [11][12]

Processor	Chip Type	CMOS Tech (μm)	Number of Point	Datapath Width (bit)	Execution Time (μs)	Frequency (MHz)	Number of Chips
Our Design	FPGA	0.13	1024	16	10.1	127	1
Xilinx FFT Core	FPGA	0.13	1024	16	41.5	100	1
Altera FFT Core	FPGA	0.13	4096	16	262	94	1
Stanford, Imagine	DSP	0.15	1024	-	20.6	180	1?
S. M. Currie FFT	ASIC	0.25	1024	16	<11	100	1?
J.-C. Kuo NTU	ASIC	0.35	1024	16	40	80	1

It's clear from Fig. 4 that the whole pipeline consists of four stages: readout of data, butterfly calculation, overflow processing and writeback of data. Notice that the writeback stage of every level overlaps with butterfly calculation stage of the next level, as presented in the hatched area of Fig. 4.

Different from the traditional method of overflow tryout with a lower efficiency, the method of overflow control implemented in this paper needs not interrupt the pipeline of calculation. Checking out the top 2 bits of writeback operands: if there exits possibility to take place overflow of result in the next level, then shifter right-shift operands 1~3 bits in the next level writeback period. If the maximum of the top 2 bits of every operands is equal to 00 in binary code, it needs 1 bit right-shift operation to prevent from overflow of lifting scheme; if the maximum is 01, it needs 2 bits right-shift(one bit is for addition overflow of 4 operands, another is for lifting scheme); if the maximum is 10 or 11 in binary code, it needs 3 bits right-shift(two bits are for the addition overflow).

The number of right-shift is added up to a register to decide the exponent value of final FFT output in the end. We observe that this approach is easy to be implemented in hardware with less right-shift operations.

IV. Simulation and Comparsion

By optimizing the mapping structure of this FFT processor, we simplify the hardware complexity and improve the performance. The architecture as described in Section III is implemented after coding in Verilog. These designs are synthesized using Xilinx ISE 5.2i on Virtex-II Pro30 FPGA. We obtain a soft-core FFT processor with the clock frequency of about 127 MHz, which can complete a 16-bit complex number of 1024-point FFT computation in 10.1 μs. Some simulation results of the FFT processor and comparison with other current fix-point FFT processors are shown in TABLE I.

From TABLE I, we observe that the FFT processor designed in this paper is better in performance than most of available FFT processors.

V. Conclusion

To develop an efficient design for FFT processor on FPGAs, we optimize the structure of memory system, simplify the hardware implementation and improve the throughput of datapath. At the same time, we propose an adaptive method of overflow control without interrupting the pipeline of calculation, which improves the efficiency of the processor with a higher calculation precision.

These designs are synthesized and implemented on Xilinx XCV2P30 FPGA chip with ISE5.2i. The clock frequency arrives 127MHz, thus this FFT processor can finish FFT calculation of 1024-point complex of 16-bit width in 10.1 μs, which is better than most of available FFT processors.

Acknowledgements

This work was supported by the National High Technology Research & Development Program of China (Grant No: 2001AA111090) and National Natural Foundation of China (Grant No: 60303017).

References

[1] B. M. Baas, "An approach to low power, high performance, fast Fourier transform processor design," [PhD Thesis], Stanford University, 1999.

[2] C. H. Sung, K. B. Lee, and C. W. Jen, "Design and implementation of a scalable fast Fourier transform core," ASIA-Pacific Conference on ASICs, pp. 295-298, 2002.

[3] L. G. Johnson, "Conflict free memory addressing for dedicated FFT hardware," IEEE Trans. Circuits and Sys. II, Vol. 39, pp. 312-316, May 1992.

[4] Y. K. Xie and B. Fu, "Design and implementation of high throughput FFT processor," Journal of Computer Research and Development, Vol. 41, No. 6, pp. 1022-1029, 2004.

[5] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," Math. of Comp., Vol. 19, No. 90, pp. 297-301, April 1965.

[6] I. Daubechies and W. Sweldens, "Factoring wavelet transforms into lifting steps," Bell Labs, Lucent Technol., Red Bank, NJ, 1996.

[7] S. Oraintara, Y. J. Chen, and T. Q. Nguyen, "Integer fast Fourier transform," IEEE Trans. Acoustics, Speech, Signal Processing, Vol. 50, No. 3, pp. 604-618, March 2002.

[8] D. Cohen, "Simplified control of FFT hardware," IEEE Trans. Acoustics, Speech, Signal Processing, Vol. ASSP-24, pp. 577-579, 1976.

[9] Y. T. Ma, "An effective memory addressing scheme for FFT processors," IEEE Trans. Signal Processing, Vol. 47, No. 3, pp. 907-911, 1999.

[10] "FFT MegaCore function user guide," Version 1.02: Altera Inc., March 2001.

[11] "High-performance 1024-point complex FFT/IFFT," Version 2.0, Xilinx Inc., July 2000.

[12] B. M. Baas, "FFT Processor Chip Info Page," available at http://www-star.stanford.edu/~bbaas/fftinfo.html, 2004.